

Monte Carlo π — Parallel Benchmark

Kystaubay Ayanat | 230103075 | 17.09.2026

Part 1: The Phantom Bug

Run	1	2	3	4	5	Average
π	2.801955	2.763208	2.718473	2.762828	2.747579	2.758809

```
=== PART 1: Phantom Bug (4 threads, 50,000,000 points, shared totalHits++)
Run 1: pi = 2.801955    (time 764.8 ms)
Run 2: pi = 2.763208    (time 773.9 ms)
Run 3: pi = 2.718473    (time 759.7 ms)
Run 4: pi = 2.762828    (time 757.9 ms)
Run 5: pi = 2.747579    (time 749.1 ms)
```

- **Result:** $\pi \approx 2.72 - 2.80$ instead of 3.1416: about 12% of hits were lost because threads overwrite each other's `totalHits++` writes (data race). The error is smaller than 1.8 – 2.4 because the VM has only one physical core.

Part 2: The Synchronization Trap

Version	π	Time (ms)	vs. 1 thread
Single-thread plain loop	3.141963	560.9	1.00x (baseline)
4 threads, synchronized	3.141710	1101.1	1.96x slower
4 threads, AtomicLong	3.141703	761.3	1.36x slower

```
=== PART 2: Synchronization Trap (50,000,000 points) ===
Single-thread plain loop : pi = 3.141963    time = 560.9 ms    (1.00x)
4 threads synchronized   : pi = 3.141710    time = 1101.1 ms    (1.96x slower)
4 threads AtomicLong     : pi = 3.141703    time = 761.3 ms    (1.36x slower)
```

- **Result:** the answer is now accurate ($\pi \approx 3.1417$), but both thread-safe versions are slower than one single thread.

Part 3: OpenMP-Style Reduction

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (T1 / TN)	Efficiency (Speedup / T)
1 (baseline)	1235.8	1.0x	100%
2	1194.1	1.03x	51.7%
4	1219.7	1.01x	25.3%
8	1225.6	1.01x	12.6%
16	1192.4	1.04x	6.5%
32	1202.7	1.03x	3.2%

=== PART 3: Reduction (100,000,000 points) ===

Threads (T)	Runtime (ms)	Speedup (T1/TN)	Efficiency (Speedup/T)	pi
1 (baseline)	1235.8	1.00x	100.0%	3.14120
2	1194.1	1.03x	51.7%	3.14147
4	1219.7	1.01x	25.3%	3.14133
8	1225.6	1.01x	12.6%	3.14173
16	1192.4	1.04x	6.5%	3.14165
32	1202.7	1.03x	3.2%	3.14147

- **Result:** π is correct for every T (3.1412 – 3.1417) and no locks are needed. Runtime stays around 1.2 s for all T: the VM gives only one physical core, so all threads share the same hardware and speedup cannot grow.

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

Speed only grows while every thread gets its own physical core. On my run 16 threads took 1192.4 ms and 8 threads took 1225.6 ms, almost the same, because the VM has just one physical core (2 logical). On an 8-core CPU the situation is the same at 16 threads: there are only 8 cores, so the extra 8 threads do not add hardware. They only share the same cores through Hyper-Threading or the OS scheduler, which splits CPU time between them. Hyper-Threads share the core's execution unit and cache, so they give much less than a real core. More threads also bring more context switches, thread start and join overhead, and more competition for cache and the memory bus. The work does not become faster; the same cores simply switch between more threads.

2. Why was the synchronized version in Part 2 slower than running on one single core?

With `synchronized` only one thread at a time can increment the counter, and a lock is taken about 39 million times (once per hit). The work becomes serial again, but each increment now also pays for lock acquire and release. When threads collide, the waiting threads are blocked and parked by the operating system, which adds context switches. The lock and the counter also move between cores' caches on every write (cache-line bouncing). A single thread has none of this overhead and increments a local variable in a CPU register. That is why `synchronized` took 1101.1 ms, 1.96x slower than one thread (560.9 ms). `AtomicLong` (761.3 ms, 1.36x slower) avoids blocking by using a CPU compare-and-swap instruction, but it still pays for cache bouncing and retries. Part 3 shows the fix: remove shared state, count locally, and combine once at the end.